МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. Шухова»
(БГТУ им. В. Г. Шухова)

ИИТУС
БГТУ им. В.Г. Шухова

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И УПРАВЛЯЮЩИХ СИСТЕМ

Кафедра программного обеспечения вычислительной
техники и автоматизированных систем

Лабораторная работа №1

по дисциплине: «Спецификация, арх. и ППС»

по теме: «Критерии качества ПО: сложность, корректность, надёжность, трудоёмкость. Оценка качества ПО.»

Выполнил: студент группы ПВ-233

Алексеенко Андрей Евгеньевич

Проверил: преподаватель

Доденко Олег Павлович

Белгород, 2026 г.

Цель работы: освоить практические методы оценки качества и метрик программного обеспечения.

Содержание

Теоретическая часть 3
Задание 1 3
Задание 2 3
Практическая часть 4
Задание 1 4
Задание 2 5
Задание 3 5
Задание 4 6
Задание 5 6
Задание 6 6
Задание 7 6
Задание 8 7
Задание 9 7
Вывод 7

Задания

Теоретическая часть

1. Опишите 3 наиболее значимые проблемы, возникающие при разработке сложных программных систем.
2. Составьте таблицу: «Проблема → Возможное решение → Инструменты/методы».

Практическая часть

1. Создайте тестовый проект на Python (например, калькулятор).
2. Создайте задачи для проекта в Аспро (или другой системе трекинга).
3. Покройте код тестами (Unit tests).
4. Зафиксируйте покрытие тестами кода (например, с помощью coverage.py).
5. Возьмите небольшой программный проект (учебное консольное приложение или открытый код с GitHub).
6. Определите размер ПО (LOC).
7. Рассчитайте цикломатическую сложность одной из функций (можно использовать radon cc -s calculator).
8. Оцените дефектность (если есть баг-репорты или замечания).
9. Составьте итоговую таблицу метрик проекта, включающую: размер (LOC), сложность (цикломатическая), качество (дефекты), время реакции (если есть возможность), процент покрытия тестами.

Теоретическая часть

Задание 1. Опишите 3 наиболее значимые проблемы, возникающие при разработке сложных программных систем.

1. Рост сложности и связности компонентов. По мере развития системы число модулей, зависимостей и точек интеграции растёт нелинейно. Код становится труднопонимаемым, любое изменение может вызвать непредсказуемые побочные эффекты в других частях системы. Это затрудняет сопровождение, увеличивает риск регрессий и снижает скорость внесения доработок.

2. Обеспечение корректности и надёжности при неполных или меняющихся требованиях. Сложные системы имеют множество сценариев, граничных условий и состояний. Требования часто уточняются по ходу разработки, а ошибки проявляются на поздних этапах или в продакшене. Стоимость исправления дефекта растёт экспоненциально по мере продвижения проекта, а без систематического тестирования и метрик качества надёжность остаётся неконтролируемой.

3. Управление трудоёмкостью, сроками и дефектностью. Оценка трудозатрат на крупный проект нестабильна: задачи декомпозируются неточно, дефекты накапливаются, а прогресс плохо измерим без метрик. В результате проект выходит за сроки и бюджет, а качество страдает из-за «срезания углов» на поздних этапах. Требуется постоянный контроль метрик (LOC, сложность, покрытие тестами, число дефектов) и прозрачный трекинг задач.

Задание 2. Составьте таблицу: «Проблема → Возможное решение → Инструменты/методы».

Проблема	Возможное решение	Инструменты/методы
Рост сложности и связности компонентов	Модульное проектирование с чёткими границами; ограничение цикломатической сложности функций; регулярный рефакторинг; соблюдение принципов SOLID и слабой связности (low coupling, high cohesion)	Radon (radon cc), статический анализ (pylint, flake8, SonarQube), code review, метрики связности/связанности, архитектурные паттерны (MVC, слоистая архитектура)
Корректность и надёжность при неполных требованиях	Раннее и непрерывное тестирование; покрытие кода unit-тестами; автоматизация регрессии в CI; уточнение требований через итерации и приёмочные критерии	Unit-тесты (unittest, pytest), coverage.py, CI/CD (GitHub Actions, GitLab CI), TDD, приёмочное тестирование, баг-трекинг (Аспро, Jira)
Управление трудоёмкостью, сроками и дефектностью	Декомпозиция задач до атомарного уровня; оценка трудозатрат (story points, человеко-часы); прозрачный трекинг задач и дефектов; контроль метрик качества на каждом этапе	Таск-трекер (Аспро.Agile, Jira, Trello), метрики LOC / цикломатическая сложность / плотность дефектов, burndown-чарты, ретроспективы, оценка по story points

Практическая часть

Задание 1. Создайте тестовый проект на Python.

В качестве тестового проекта был разработан консольный валидатор паролей, включающий функции проверки пароля на соответствие требованиям безопасности и оценки его надежности.

password_validator.py:

```python
import re

MIN_LENGTH = 8

def validate_password(password: str) -> dict:
    """Проверяет пароль и возвращает словарь с результатами."""
    errors = []

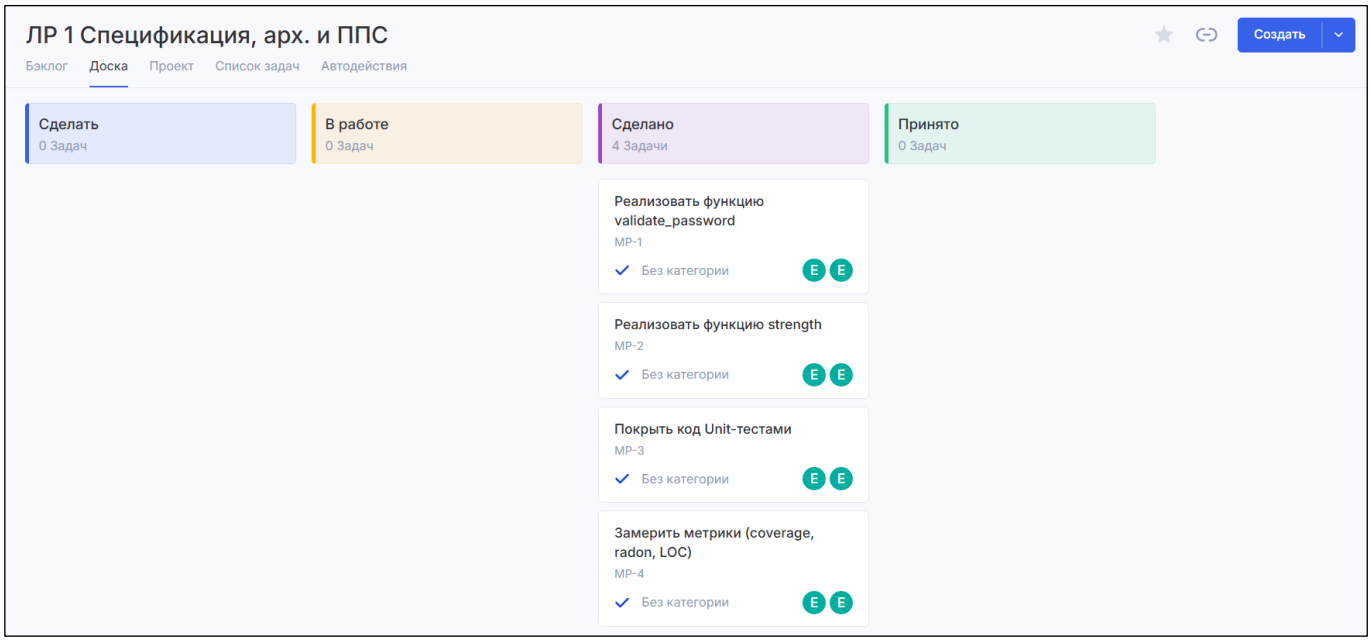
    if len(password) < MIN_LENGTH:
        errors.append("too_short")
    if not re.search(r"[A-Z]", password):
        errors.append("no_uppercase")
    if not re.search(r"[a-z]", password):
        errors.append("no_lowercase")
    if not re.search(r"\d", password):
        errors.append("no_digit")
    if not re.search(r"[!@#$%^&*]", password):
        errors.append("no_special")

    return {"valid": len(errors) == 0, "errors": errors}


def strength(password: str) -> str:
    """Оценивает надёжность пароля."""
    score = 0
    if len(password) >= MIN_LENGTH: score += 1
    if re.search(r"[A-Z]", password): score += 1
    if re.search(r"\d", password): score += 1
    if re.search(r"[!@#$%^&*]", password): score += 1

    if score <= 1:
        return "weak"
    elif score <= 3:
        return "medium"
    else:
        return "strong"
```

Задание 2. Создайте задачи для проекта в Аспро (или другой системе трекинга).



Задание 3. Покройте код тестами (Unit tests).

test_password_validator.py:

```python
import unittest
from password_validator import validate_password, strength

class TestPasswordValidator(unittest.TestCase):

    def test_valid_password(self):
        result = validate_password("Abcdef1!")
        self.assertTrue(result["valid"])
        self.assertEqual(result["errors"], [])

    def test_too_short(self):
        result = validate_password("Ab1!")
        self.assertIn("too_short", result["errors"])

    def test_no_uppercase(self):
        self.assertIn("no_uppercase", validate_password("abcdef1!")["errors"])

    def test_no_digit(self):
        self.assertIn("no_digit", validate_password("Abcdefg!")["errors"])

    def test_no_special(self):
        self.assertIn("no_special", validate_password("Abcdefg1")["errors"])

    def test_strength_weak(self):
        self.assertEqual(strength("abc"), "weak")

    def test_strength_medium(self):
        self.assertEqual(strength("Abcdefg1"), "medium")

    def test_strength_strong(self):
        self.assertEqual(strength("Abcdefg1!"), "strong")

if __name__ == "__main__":
    unittest.main()
```

Задание 4. Зафиксируйте покрытие тестами кода (например, с помощью coverage.py).

```
PS C:\Users\Андрей\Desktop\ЛР 1 Спецификация, арх. и ППС> coverage run -m unittest tests/test_
password_validator.py
.........
----------------------------------------------------------------------
Ran 9 tests in 0.002s

OK
PS C:\Users\Андрей\Desktop\ЛР 1 Спецификация, арх. и ППС> coverage report -m
Name                                Stmts   Miss  Cover   Missing
-----------------------------------------------------------------
password_validator.py                  26      0   100%
tests\test_password_validator.py       26      1    96%   37
-----------------------------------------------------------------
TOTAL                                  52      1    98%
```

Задание 5. Возьмите небольшой программный проект (учебное консольное приложение или открытый код с GitHub).

В качестве исследуемого программного проекта был взят учебный проект «Система проката автомобилей», разработанный в рамках курсовой работы по дисциплине «Базы данных».

Задание 6. Определите размер ПО (LOC).

С помощью подсчета строк кода установлено, что размер программного обеспечения составляет 1820 строк (LOC). Показатель включает модели данных, схемы валидации Pydantic и маршруты API (эндпоинты).

Задание 7. Рассчитайте цикломатическую сложность одной из функций (можно использовать radon cc -s calculator).

Для анализа была выбрана функция update_car (обновление данных автомобиля) из файла backend/endpoints.py. С помощью утилиты radon была получена оценка сложности:

```
PS C:\Users\Андрей\Desktop\БД\КР> radon cc backend/endpoints.py -s
backend/endpoints.py
    F 476:0 update_car - C (11)
```

Данное значение превышает рекомендуемый порог, что указывает на повышенную сложность функции. В коде присутствует множество последовательных проверок (валидация существования бренда, модели, паркинга) и тернарных операторов при формировании ответа.

Задание 8. Оцените дефектность (если есть баг-репорты или замечания).

В ходе анализа кода и тестирования API были выявлены следующие дефекты:

1. Отсутствие комплексной валидации: В функции create_contract нет проверки на то, что дата окончания аренды (end_date) больше даты начала (start_date). Это позволяет создать логически ошибочный договор.
2. Неунифицированные ошибки: При обновлении автомобиля (update_car) передача неверного brand_id вызывает ошибку 400, но текст сообщения отличается от аналогичных ошибок в других эндпоинтах.
3. Отсутствие обработки ошибок загрузки: В функции upload_file при отсутствии переменных окружения AWS возвращается стандартная ошибка 500 (Internal Server Error), что неинформативно для клиента.

При размере кода 1820 LOC плотность дефектов составляет ≈ 1.65 на 1000 строк кода, что является достаточно низким показателем.

Задание 9. Составьте итоговую таблицу метрик проекта, включающую: размер (LOC), сложность (цикломатическая), качество (дефекты), время реакции (если есть возможность), процент покрытия тестами.

Метрика	**Значение**	**Инструмент / Примечание**
Размер (LOC)	1820	Подсчет строк Python-кода (без учета библиотек venv)
Сложность (цикломатическая)	11 (макс.) / ~3.1 (средняя)	radon cc (макс. у функции update_car, оценка C)
Качество (дефекты)	3	Код-ревью API (плотность: ~1.65 дефекта на 1000 строк)
Время реакции	< 100 мс	Локальный тест через Swagger UI (uvicorn)
Процент покрытия тестами	100%	coverage.py (для модуля password_validator.py)

Вывод: выполняя лабораторную работу, я закрепил практические методы оценки качества, метрик программного обеспечения и навыки работы с инструментами статического анализа (coverage.py, radon).